Table of Contents

cc-orchestration — Install Guide

Version 3.2.1 · Updated 2026-05-24

A policy + glue layer on top of the official Claude Code plugin ecosystem. The official plugins do the heavy lifting (TDD enforcement, code-review, browser automation, doc lookups, GitHub API). This plugin enforces *your* engineering policy on top of them: pre-implementation discipline, plan review cycles, architecture enforcement, no-assumptions rule, official-docs-first, assumption-validation tests, comment policy, PR merge-conflict awareness.

Critical principle: official plugins are tools the custom agents *use*. They never replace the custom agents.

---

1. Quick install (30 seconds, any OS)

Inside Claude Code:

```
/plugin marketplace add valverdesolera/cc-orchestration
/plugin install claude-code-orchestration@cc-orchestration
/reload-plugins
```

Verify:

```
/agents   →  engineering-orchestrator + 23 specialists
/hooks    →  5 hooks
/mcp      →  context7 (and any others you've added)
```

Then ask Claude:

> "What does CLAUDE.md say about UNVERIFIED claims and canonical per-dependency documentation?"

If it returns §3 (verification-status rule) and §15 (<CanonicalName>Documentation.md convention), you're live.

---

2. Prerequisites

Tool	Version	Install
Claude Code	2.1.143+	npm install -g @anthropic-ai/claude-code or via Homebrew

git	2.30+	macOS: pre-installed or brew install git. Windows: https://git-scm.com/download/win (also installs Git Bash). Linux: apt install git / dnf install git.
bash	Any	macOS/Linux: pre-installed. Windows: use **Git Bash** (ships with Git for Windows). Do NOT try to use WSL just for this.
Python 3.10+	Optional	Only needed if you run the per-repo bootstrap. Install: macOS brew install python3, Linux apt install python3 / dnf install python3, Windows winget install Python.Python.3.12.

Confirm before installing:

```
claude --version    # 2.1.143 or later
git --version       # 2.30 or later
bash --version      # any
python3 --version   # 3.10 or later (only needed for bootstrap)
```

---

3. Optional: per-repo overlay (the bootstrap)

The plugin gives you all the agents/skills/hooks/commands. The bootstrap adds the per-repo layer that plugins *cannot* install:

- docs/ignored/{implementation,workbooks,context}/ folders for transient artifacts (kept out of git via .git/info/exclude)
- Git hooks that block AI attribution, env files, and private artifacts from commits/pushes
- Branch and worktree wrapper scripts that respect your conventions
- Cleanup of stale personal config entries (microsoftLearn MCP duplicate) in ~/.claude.json (with backup)

Run it once per repo where you want the overlay. **You do not need this for the plugin to work** — only run it if you want the per-repo file/hook layer.

Two ways to run it

Without cloning (recommended for first-time setup on a new machine):

```
cd /path/to/your/repo
curl -fsSL https://raw.githubusercontent.com/valverdesolera/cc-orchestration/main/bootstrap/install_repo_bootstrap.sh | bash -s -- --yes
```

If you have the repo cloned locally:

```
cd ~ && git clone https://github.com/valverdesolera/cc-orchestration.git    # one-time
bash ~/cc-orchestration/bootstrap/install_repo_bootstrap.sh --repo /path/to/your/repo --yes
```

The bootstrap is idempotent — safe to re-run.

Verify the bootstrap

```
cd /path/to/your/repo
ls docs/ignored/                    # implementation/  workbooks/  context/
git config --get core.hooksPath     # path under ~/.local/share/cc-orchestration/
cat .git/info/exclude | head -20    # personal-only patterns
```

---

4. Recommended companion plugins

This plugin is designed to compose with the official Anthropic plugin set. To see the canonical install commands (read from a single source of truth so they cannot drift):

```
bash ~/cc-orchestration/bootstrap/install_repo_bootstrap.sh --print-plugins
```

This prints /plugin install … lines for each recommended official plugin. **Paste them into Claude Code one line at a time** — pasting all at once concatenates them and Claude Code reports "Malformed input."

The 15 recommended plugins:

Plugin	Why it matters
context7	Version-aware library docs (the generic doc-lookup fallback)
code-review	/code-review runs the diff-level bug pass in the implementation feedback loop
feature-dev	code-explorer + code-architect for greenfield design
superpowers	TDD red-green-refactor enforcement
github	GitHub API for PR creation, code scanning, GraphQL
atlassian	Jira + Confluence APIs
playwright	Browser automation + E2E testing
chrome-devtools-mcp	Live DevTools control for frontend debugging

serena	Semantic, LSP-backed code retrieval (preferred for symbol lookups)
claude-md-management	Audit CLAUDE.md quality, capture learnings
commit-commands	Safe /commit flow
claude-code-setup	Per-stack setup helpers
pr-review-toolkit	Multi-angle PR review
microsoft-docs	Microsoft / Azure / .NET docs (preferred over context7 for MS stack)
frontend-design	UI component pattern matching

Each has a documented fallback_when_missing in reference/recommended-plugins.json — agents detect missing plugins and degrade gracefully (e.g., custom code-reviewer runs without the /code-review bug pass and labels the result as "partial").

Additionally, six MCPs without official plugins yet are documented in the same reference file (Postman, codegraphcontext, tree-sitter, codanna, codeql, srclight). Install manually via /mcp add when you need them.

---

5. Updating

Inside Claude Code, run the two built-in commands:

```
/plugin marketplace update cc-orchestration
/plugin update claude-code-orchestration@cc-orchestration
```

After that, optionally verify the result:

```
/cco-update
```

/cco-update is a **verifier**, not an updater. Claude Code's plugin system doesn't allow custom slash-commands to invoke built-in slash-commands like /plugin update, so a custom command can't perform the update on your behalf. What /cco-update does is read the installed plugin version, the local marketplace HEAD, and the GitHub remote, then report whether all three agree. If you're out of date, it tells you the exact commands to run (the two above).

If /plugin marketplace update returns "(no content)" or the install version doesn't change after /plugin update, you're probably on a local-path marketplace registration (/plugin marketplace add ~/cc-orchestration) which doesn't reliably propagate file changes. Use the full re-install sequence instead:

```
/plugin uninstall claude-code-orchestration@cc-orchestration
/plugin marketplace remove cc-orchestration
/plugin marketplace add valverdesolera/cc-orchestration
```

```
/plugin install claude-code-orchestration@cc-orchestration
/reload-plugins
```

This switches you to the GitHub URL form of the marketplace, which behaves correctly for future updates.

On a machine where you cloned ~/cc-orchestration locally (and need the bootstrap script + reference files updated too), also run `cd ~/cc-orchestration && git pull`. This is unrelated to the plugin update but worth doing if you'll be re-running the bootstrap on additional repos.

To also update the bootstrap script's effect across repos you've already bootstrapped, re-run the bootstrap inside each repo — it's idempotent and refreshes the hook templates.

---

6. Per-OS notes

macOS

Works out of the box. If you use Homebrew, install git and python via brew. The bootstrap uses `python3` directly; macOS has it as `/usr/bin/python3`.

Linux

Works out of the box. Use your distro's package manager for git and python3.

Windows

Use **Git Bash** (ships with Git for Windows), not PowerShell, for any bootstrap or shell commands.

- Typing `bash` in PowerShell routes to WSL, which is a *different* environment and is usually not installed. Open Git Bash as its own application (Windows key → "Git Bash") instead.
- Paths use `/c/Users/your-name/…` form in Git Bash to refer to `C:\Users\your-name\…`. Folder names with spaces need quotes.
- The bootstrap auto-detects whether `python3` or `python` is available and uses whichever works. If neither is found, install Python via `winget install Python.Python.3.12`, then close and reopen Git Bash so PATH refreshes.
- The Microsoft Store stubs for `python.exe` / `python3.exe` can intercept Python calls and launch the App Store. Disable them: **Settings → Apps → Advanced app settings → App execution aliases → toggle off python.exe and python3.exe**.
- If your repo lives in an OneDrive-synced folder, OneDrive can briefly lock files during the bootstrap. If anything fails mid-run, pause OneDrive sync (system tray icon → pause) and re-run.

Inside Claude Code itself, the marketplace and plugin commands are identical to macOS/Linux.

7. Troubleshooting

/plugin install returns "Malformed input"

You pasted multiple /plugin install lines at once. Paste each one separately.

claude command not found after reinstall

Stale npm cache. Fix:

```
rm -rf ~/.nvm/versions/node/*/lib/node_modules/@anthropic-ai/claude-code
2>/dev/null
rm -rf ~/.claude-code-* 2>/dev/null
npm install -g @anthropic-ai/claude-code
hash -r   # bash; for zsh use: rehash
```

bash: /opt/homebrew/bin/brew: No such file at shell startup (macOS)

Your ~/.zprofile or ~/.bashrc is sourcing Homebrew but Homebrew isn't installed at that path. Either install Homebrew (/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)") or comment out the line that references it.

bad pattern: # (zsh on macOS)

Zsh treats # as a glob unless you opt in to bash-style comments. Add to ~/.zshrc:

```
setopt interactivecomments
```

Then source ~/.zshrc. After that, pasted multi-line shell with comments works correctly.

Bootstrap on Windows says "Python 3 is required but was not found"

Install Python (winget install Python.Python.3.12), close and reopen Git Bash, then re-run. If python works in Git Bash but python3 doesn't, the bootstrap will try to auto-create a python3.exe shim alongside python.exe — and tell you it did. If that auto-shim fails (permission denied), create it manually:

```
PYDIR="$(dirname "$(command -v python)")"
cp "$PYDIR/python.exe" "$PYDIR/python3.exe"
```

Then re-run the bootstrap.

PowerShell shows the WSL error when I type bash …

Same issue as above. Don't use bash from PowerShell. Open Git Bash as a separate application and run shell commands from there.

Plugin loads but agents/skills don't appear

Run /reload-plugins inside Claude Code. If that doesn't help, uninstall and reinstall:

```
/plugin uninstall claude-code-orchestration@cc-orchestration
/plugin install claude-code-orchestration@cc-orchestration
```

CLAUDE.md rules don't seem to be applied

Confirm CLAUDE.md is actually loaded: in a fresh Claude Code session, ask "What's in CLAUDE.md §3?" — you should get back the official-docs-first routing table. If you get a generic answer, the plugin may not be installed in the active project; run /plugin to confirm.

Pre-commit hook blocks a legitimate commit (false positive)

The comment-policy regex may flag string literals that look like ticket IDs. To bypass for a single commit:

```
ALLOW_COMMENT_POLICY_BYPASS=1 git commit -m "..."
```

Other bypass envs: ALLOW_CLAUDE_ARTIFACT_COMMIT=1, ALLOW_ENV_COMMIT=1. Use them sparingly and only when you've verified the match is a false positive.

GitHub auth fails when cloning on a new machine

The repo is public — no auth needed for read. If you see auth errors, you're likely using SSH (git@github.com:…) without keys configured. Either set up SSH keys or use HTTPS (https://github.com/…). The bootstrap also globally rewrites SSH URLs to HTTPS via git config --global url."https://github.com/".insteadOf "git@github.com:" — apply that manually if needed.

Worktree wrappers don't copy my expected files

Edit .worktreeinclude in the repo root. One pattern per line, gitignore-style. Defaults: docs/ignored/, .env, .env.*, .envrc.

---

8. Uninstall

Inside Claude Code:

```
/plugin uninstall claude-code-orchestration@cc-orchestration
/plugin marketplace remove cc-orchestration
```

To also remove the per-repo bootstrap from a specific repo:

```
cd /path/to/repo
git config --unset core.hooksPath
# Optionally remove docs/ignored/ (you may want to keep its contents):
```

```
# rm -rf docs/ignored
# Remove personal patterns added to .git/info/exclude (edit manually).
```

To remove the shared state directory and wrapper scripts:

```
rm -rf ~/.local/share/cc-orchestration
```

~/.claude.json is left as-is. The pre-cc-orchestration backup remains at ~/.claude.json.backup-pre-cc-orchestration if you ever need to restore.

---

9. Multi-machine setup

The plugin and its updates flow via the public GitHub repo. On each machine:

```
/plugin marketplace add valverdesolera/cc-orchestration
/plugin install claude-code-orchestration@cc-orchestration
```

To pull updates later, run /plugin marketplace update cc-orchestration and /plugin update claude-code-orchestration@cc-orchestration inside Claude Code (see §5 for the full update flow and the fallback re-install sequence).

No SSH keys, no PATs, no collaborator invites — the repo is public, the marketplace handles the fetch. You can have the plugin on as many machines as you want without any account-linking concerns.

For per-machine git identity hygiene when using the plugin in personal vs work repos, set user.email per repo (not globally) so personal repos use your personal identity and work repos use your work identity:

```
cd /path/to/personal/repo
git config user.email "your-personal@email.com"

cd /path/to/work/repo
# leaves the global default in place (your work email)
```

---

10. Versioning

This plugin uses semantic versioning. Version is declared in three places (kept in sync by CI):

- plugins/claude-code-orchestration/.claude-plugin/plugin.json → version
- .claude-plugin/marketplace.json → metadata.version and plugins[0].version
- docs/REQUIREMENTS_SPEC.md → version header

When the plugin version bumps and is pushed to main, the GitHub Actions workflow .github/workflows/auto-tag.yml automatically creates and pushes a matching vX.Y.Z tag. No manual git tag needed.

11. Where to go next

- README.md — landing page with the 30-second install
- REQUIREMENTS_SPEC.md — full requirements spec (functional requirements, acceptance criteria, traceability matrix, guardrails)
- plugins/claude-code-orchestration/CLAUDE.md — the engineering ruleset loaded into every Claude Code session
- plugins/claude-code-orchestration/reference/recommended-plugins.json — single source of truth for plugin dependencies + fallbacks